Battleship Game - Requirements Analysis Document

1. Project Summary

1.1 Background

Battleship is a classic two-player strategy based guessing game. The game is played with four 10x10 grids where each square is identified by a coordinate; one letter (A to J) and one number (1 to 10). Each player uses two grids; one for placing ships and recording enemy attacks, and another for recording the player's own attacks. Before a game begins, each player secretly arranges their fleet of ships on their primary grid. A ship occupies a select number of consecutive squares and can be oriented horizontally or vertically. The number of squares that a ship occupies is dependent on the ship type. Ships cannot overlap.

The game begins once both players have placed all of their ships. Players alternate taking turns by guessing a square on their enemy's primary grid. Guesses are identified by the coordinate corresponding to the square (ex: E7). If a player guesses the correct location of an enemy ship they are awarded a "hit". If not, the player receives a "miss". The result of a player's attack is recorded on their secondary grid. Players continue to alternate taking turns. A ship is destroyed if all squares that it occupies are hit. The first player to destroy all of their enemy's ships wins the game.

1.2 Purpose

The purpose of our project is to upgrade the classic game of Battleship. The goal is to add exciting contemporary features to the game, providing Battleship players with a new experience while still enjoying an old favourite. Users will have the opportunity to play against the computer (with 2 difficulty levels) or pass and play with a friend. This project brings creative new aspects to the game such as different maps to choose from, impressive power-ups, different fleet layouts, and animations to celebrate successful hits. The user will play the entire game through the GUI; there will be no need for a command-line inter face.

1.3 Scope

There are multiple features that need to be implemented for the project. Many of these features are essential for a basic game of Battleship, and some are additional elements that have been included to enhance the user's experience.

Standard Low-Level Features

These features are crucial to the development of the game and must be completed first. They consist of:

- a simple GUI which allows the user to play the game
- appealing graphics
- a player vs computer feature
- mouse inputs
- key inputs

Additional Features

These features are necessary to ensure an upgraded game and elevated experience from the original Battleship. They include:

- a player vs player feature (pass and play)
- a power-up and points system
- a high score feature
- different fleet options
- different map options
- an instructions menu feature, which includes
    - an instructions tab
    - power-up descriptions
    - an exit game option

Time Permitting Features

These features have been conceived as part of a "stretch goal". If the project is running ahead of schedule, the team will look into implementing a selection of the following features. They include:

- playing the game in windowed or fullscreen mode (window scaling)
- providing the user with the ability to save and return to a game
- sound effects for hits and misses
- giving the user the ability to design/import their own maps

1.4 Target Users

The target demographic is anyone interested in playing a computer-based game of Battleship, primarily children and teenagers.

1.5 Objectives & Success Criteria

In order to successfully implement a battle ship game the following objectives must be completed.

Table 1: Task list of project objectives

Project Objectives
☐ Implement a player vs. computer game mode
☐ Implement a player vs. player game mode
☐ Implement a point and power-up system
☐ Implement three different game maps
☐ Implement three different fleet options
☐ Develop a fully functioning bug free system
☐ Develop an efficient and challenging enemy computer algorithm
☐ Develop a fully functioning GUI
☐ Develop an aesthetically pleasing game
☐ Optimize the game for both Mac and PC operating systems
☐ Finish development by the end of the fall 2021 term

2. System Description

2.1 Definitions

User: the individual playing the game

Map: a grid of squares on which the game is played

Coordinate: a letter and a number identifying an individual grid square

Ship: a sequence of squares occupied by a player

Fleet: a collection of ships

Attack: a guess of the coordinate location of an enemy ship

Hit: a successful player attack

Miss: an unsuccessful player attack

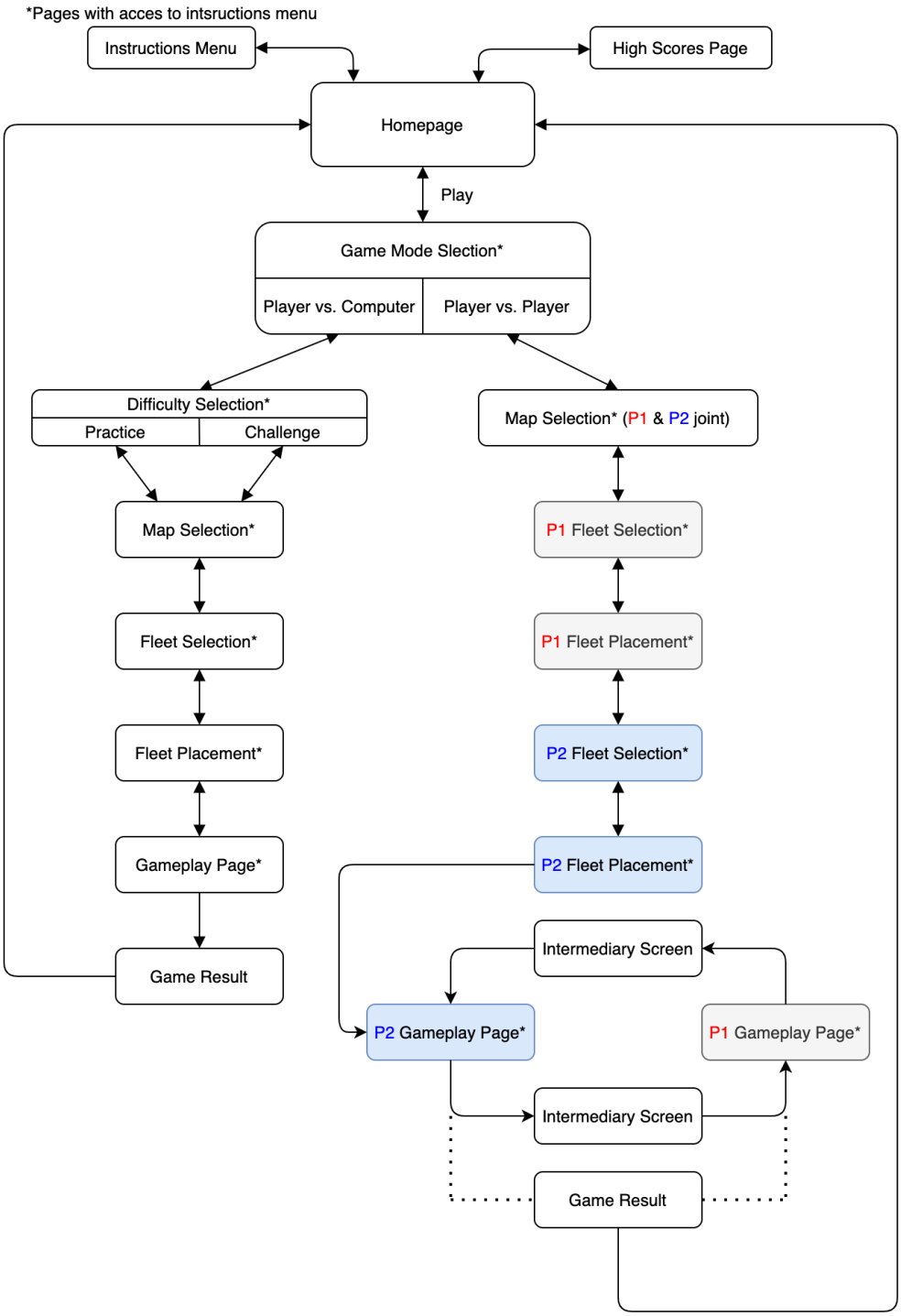
Power-up: an ability that changes the functionality of an attack

GUI: graphical user interface

2.2 Layout

The following flowchart represents the page layout of the system and how it will be navigated by the user.

Figure 1: Flowchart representing system layout



2.3 Parameters

Power-ups

In game, players can buy power-ups using points obtained by taking turns, hitting enemy ships, and sinking enemy ships. These power-ups will vary in ability, and different power-ups will be available to each player based on the fleet they select. Power-ups can only be used once. The advantage that they provide is instantaneous and takes effect on the current round.

Map Selection

As part of our game enhancements, 2 different maps will be provided for selection when starting a new game, in addition to the classic 10x10 grid. This allows for significantly more varied games, as each map will have distinct layouts allowing for diverse ship placements.

Fleet Selection

Different fleets can be selected at the start of each game. They will consist of ships with different shapes and sizes to make locating the enemy more challenging.

2.4 Inputs & Outputs

Mouse inputs

- User can start a new game.
- User can interact with the game's GUI to play by:
    - Targeting a square on the opponent's map for attack
    - Selecting and using power-ups
    - Opening the instructions menu to find game rules
    - Dragging-and-dropping, then orienting ships onto the map before the game starts
- User can navigate to the instructions menu in order to review the rules
- User can choose to cancel the current game at any time.
- User can close the program at any time.

Keyboard inputs

- When dragging-and-dropping ships onto the map, the spacebar can be used to orient the ship vertically or horizontally
- Ability to enter a username to record high scores
- Targeting a square on the opponent's map for attack with arrow keys

Screen outputs

- The program will output a GUI to the screen
- The GUI will show:
    - Different pages such as a homepage, gameplay page, and high scores page
    - Pages will display data to represent the game, including images, and text
    - A popup window will display the winner of a game

Speaker outputs (Stretch Goal)

- The program will output sound effects, including:
    - A "hit" sound when an attack hits a ship
    - A "miss" sound when an attack misses
    - An "explosion" sound when a ship is destroyed
    - A "win" or "lose" sound when the player wins or loses

Data inputs/outputs

- User scores are outputted to a file/database if they meet high score requirements
- High scores are read from a file/database when the user checks the high scores

There are two groups of high scores:

1. Local: high scores for this device only; stored in a file in the application folder
2. Global: high scores for all users on any device, stored in a database (updates with a valid internet is connected)

2.5 Assumptions & Constraints

Assumptions:

- Twice-weekly team meetings will continue
- All developers will be available for at least 5 hours of work per week
- Each developer has prior knowledge of software development practices
- Developers will follow the agreed-upon methods of software development and programming standards
- Any C++ library function can be used

Constraints:

- The software must be completed by the end of the 2021 Fall term
- Developers cannot be expected to work an excessive amount of time each week
- Developers will not be expected to purchase any software to run the project
- The project is constrained to the C++ language

3. Requirements

The following is a list of all functional and non-functional requirements associated with the project.

3.1 Functional Requirements

Fundamental (Player vs. Computer)

- As a user, I want a player vs. computer game mode so that I can play alone.
- As a user, I want to be able to interact with the GUI using a keyboard, mouse, and trackpad.
- As a user, I want to select the placement of my fleet so that I can create unique strategies.
- As a user, I want the GUI to display the current state of the game.
- As a user, I want animations when placing an attack for an engaging gameplay experience.
- As a user, I want the GUI to display when my attack has missed.
- As a user, I want the GUI to display when my attack has hit.
- As a user, I want animations when a ship is destroyed for an engaging gameplay experience.
- As a user, I want the GUI to display what enemy ships I have sunk, and what enemy ships I have left to sink.
- As a user, I want the option to surrender and quit the game in player vs. computer mode.
- As a user, I want a popup to indicate if I won the game.

Player vs. Player

- As a user, I want multiple difficulty options so that I can be sufficiently challenged (practice difficulty & challenge difficulty).
- As a user, I want a local player vs. player game mode so that I can play with a friend (pass and play).
- As a user, I want an intermediary page between player vs. player turns to ensure fair play.
- As a user, I want the game to keep track of individual game states when switching between players.
- As a user, I want the option to surrender and quit the game in player vs. player mode.
- As a user, I want a popup to indicate which player won the game.

Point System

- As a user, I want to gain points as a reward for taking turns, hitting enemy ships, and sinking enemy ships.
- As a user, I want to buy power-ups using points so that I can gain a competitive advantage.
- As a user, I want the GUI to display my points so that I can plan my attack strategy.

Fleet & Map

- As a user, I want to choose between different fleet layouts for greater gameplay variety (3 fleet layouts).
- As a user, I want fleet-specific power-ups so that I can create unique strategies.
- As a user, I want multiple game maps with different grid layouts for greater gameplay variety (3 maps).

Additional

- As a user, I want an instructions menu in case I get stuck.
- As a user, I want the ability to return to a previous page in case I make a wrong selection.
- As a user, I want a random fleet placement button for quick play.
- As a user, I want the game to store high scores (the fewest number of turns to win).

3.2 Non-Functional Requirements

- As a user, I want the game to run on Windows and Mac operating systems for more flexibility.
- As a user, I want the game to run on 4GB of RAM so that I can play on any laptop.
- As a user, I want the game to be intuitive so it is easy to learn.
- As a user, I want to have turns registered in at most 2 seconds so that the game progresses quickly.
- As a user, I want the game to run within 10 seconds of startup.
- As a user, I want the game to have an appealing art style.
- As a developer, I want the game to be built with proper architecture and standards so that it can be updated and maintained easily.

4. Prototypes of GUI Interface

The following images are prototypes of all GUI interfaces.

Figure 2: Homepage

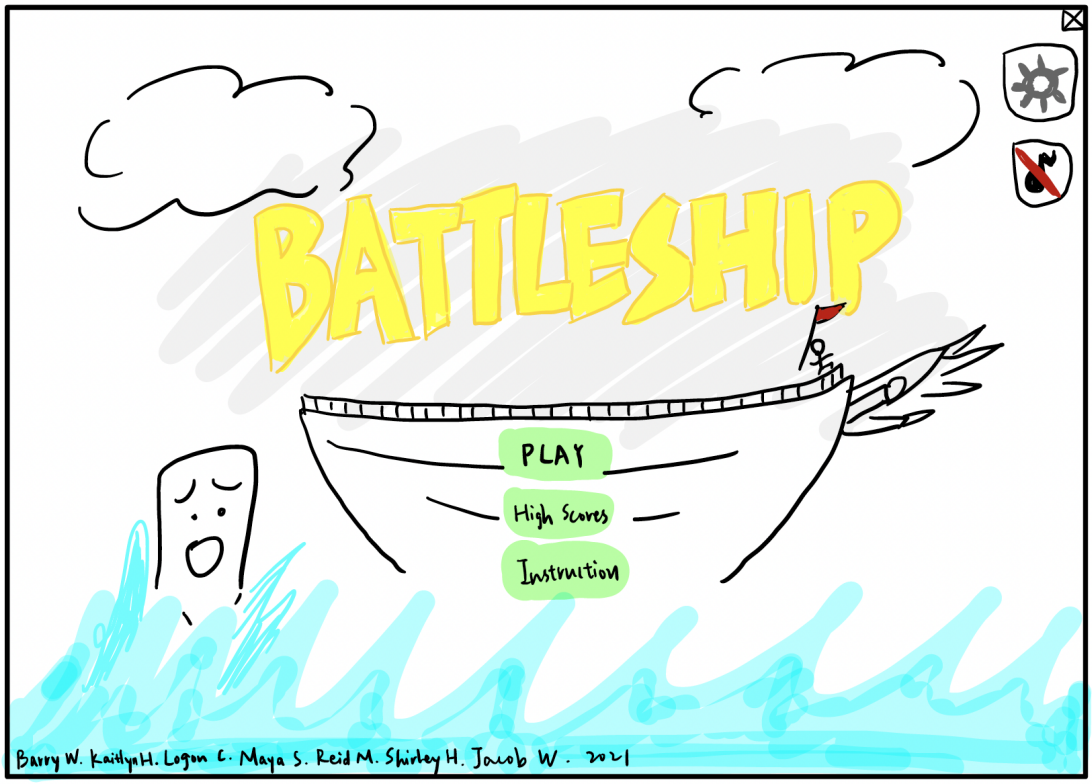


Figure 3: Instructions menu

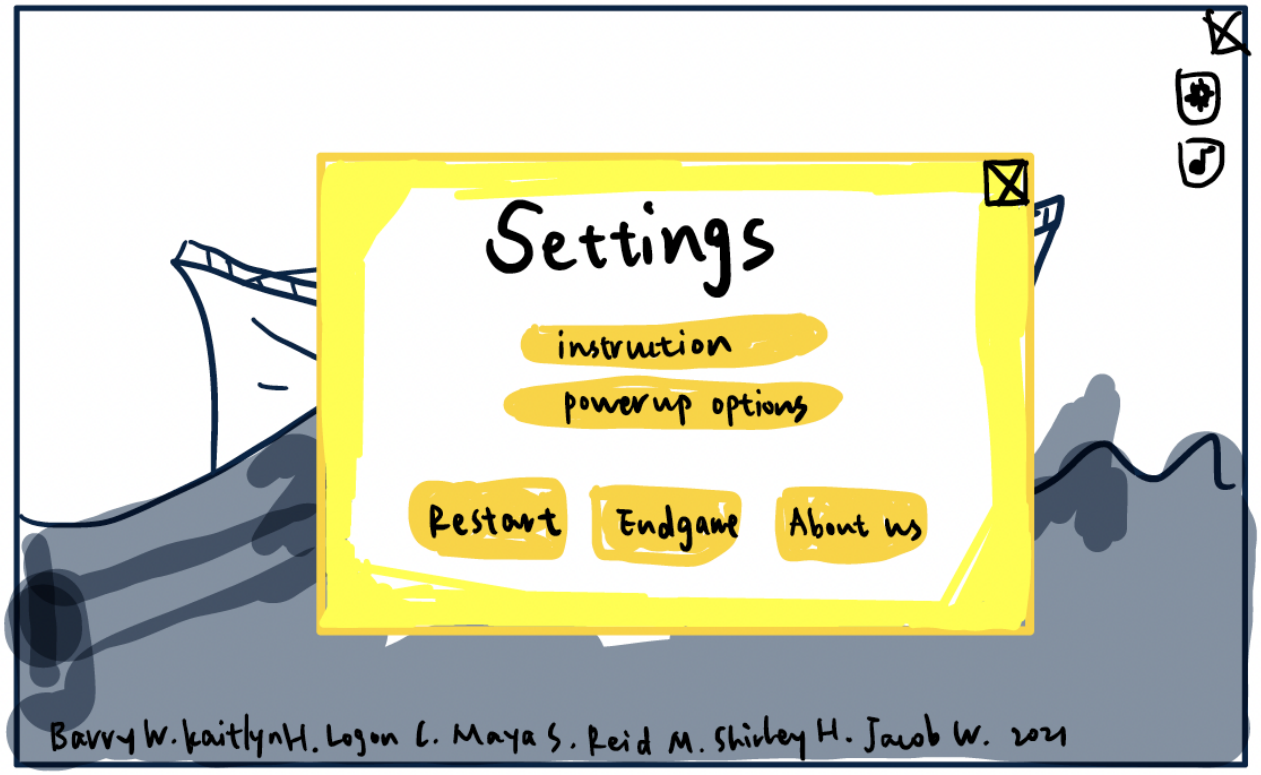


Figure 4: Game mode selection page

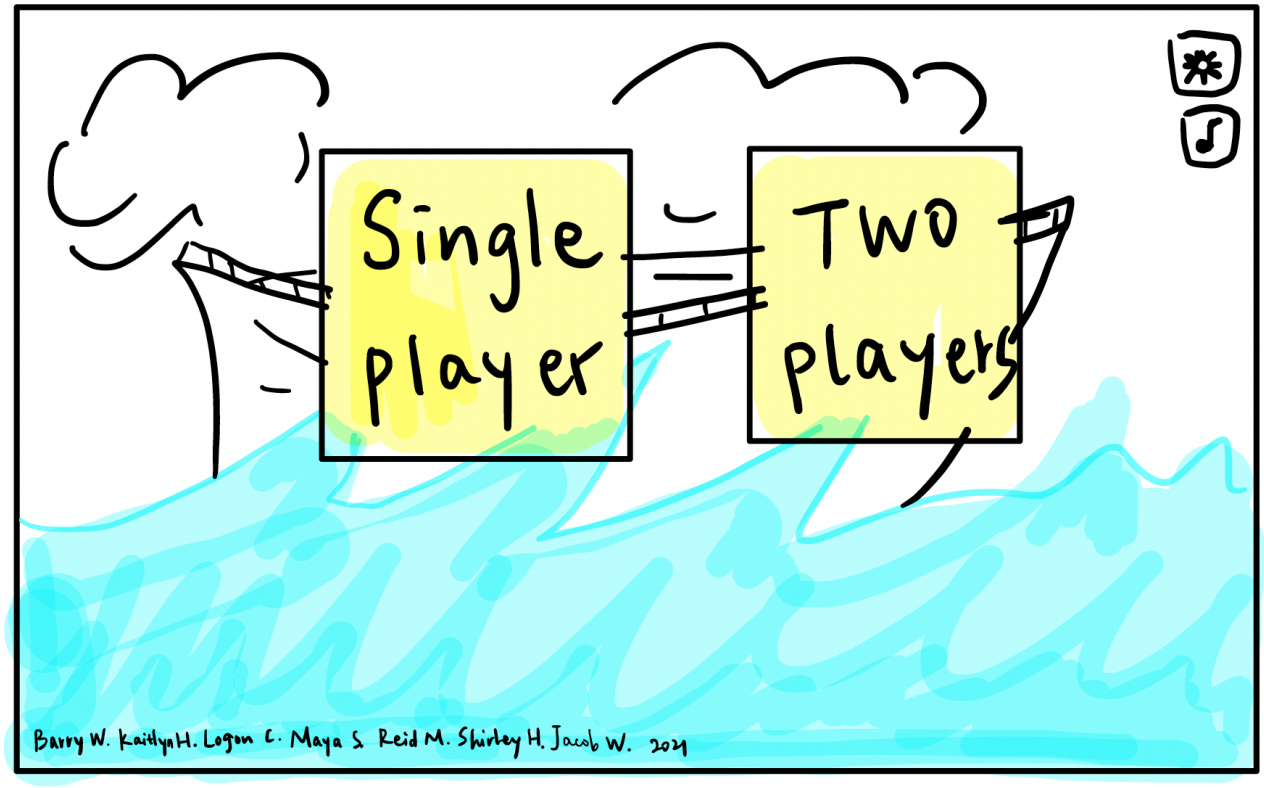


Figure 5: Player vs. computer difficulty selection page

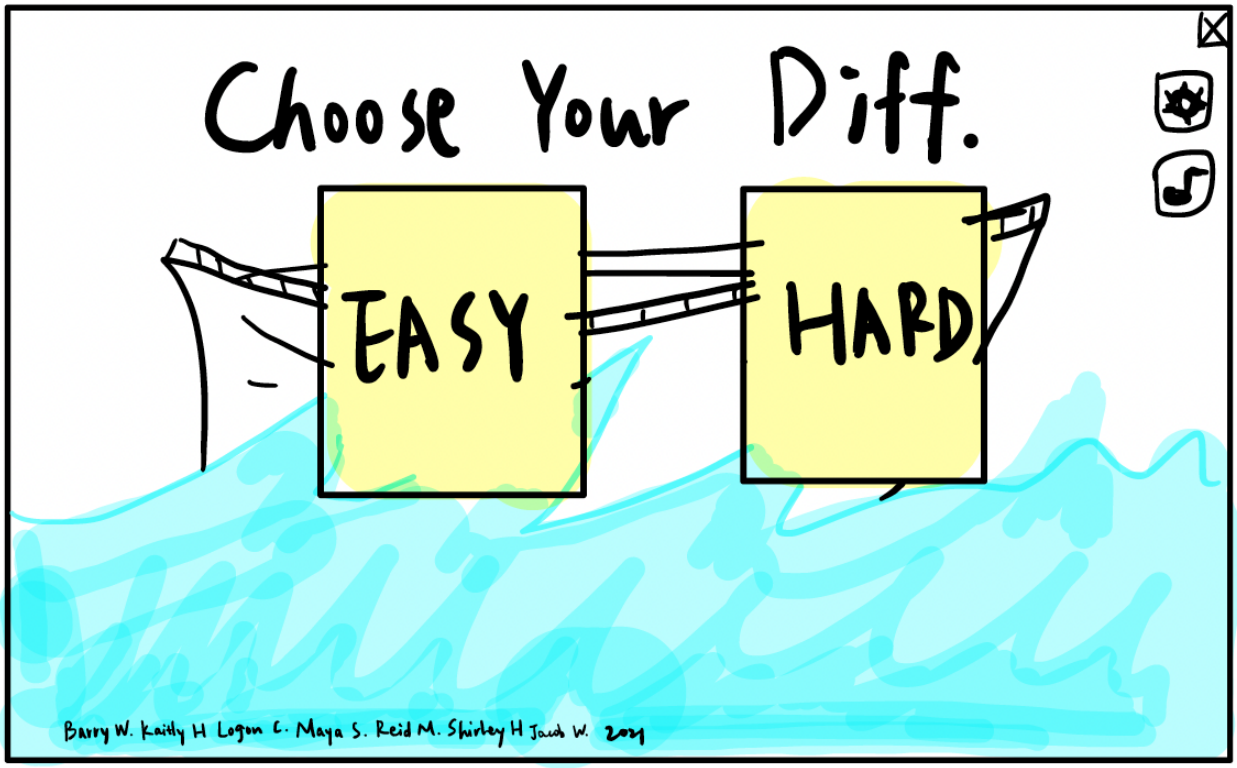


Figure 6: Map selection page

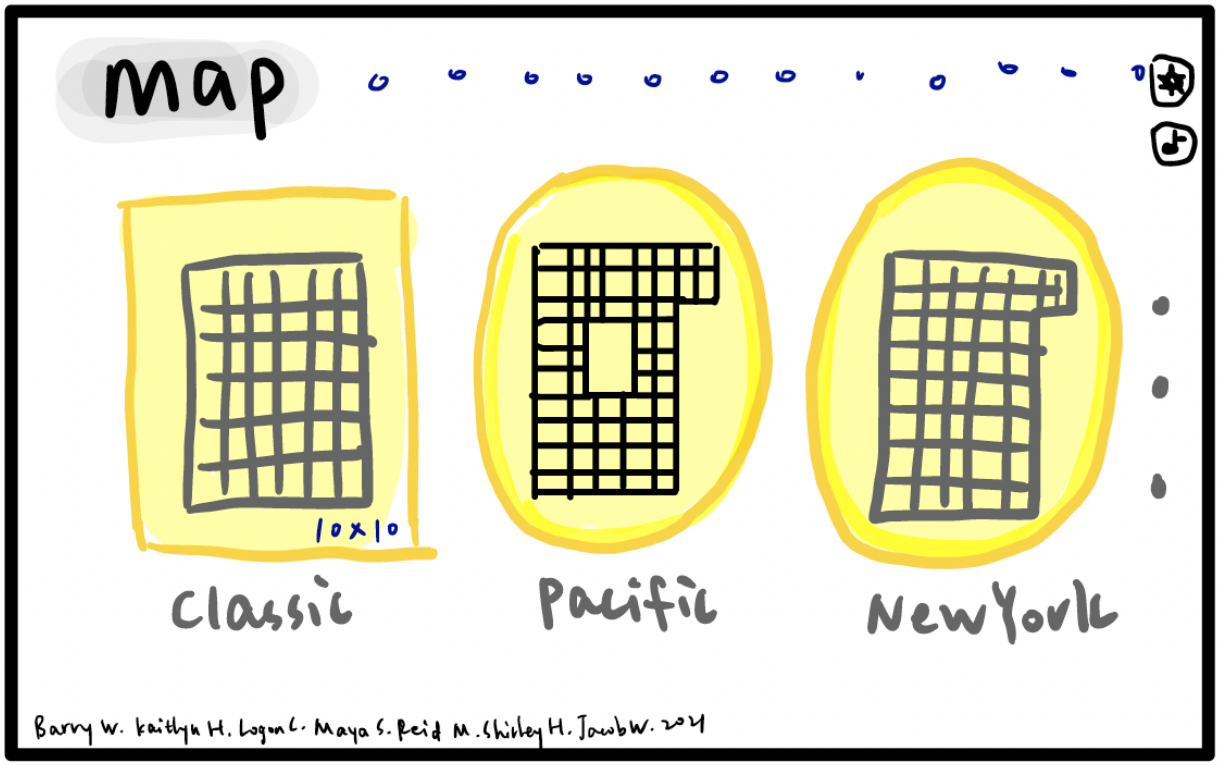


Figure 7: Fleet selection page

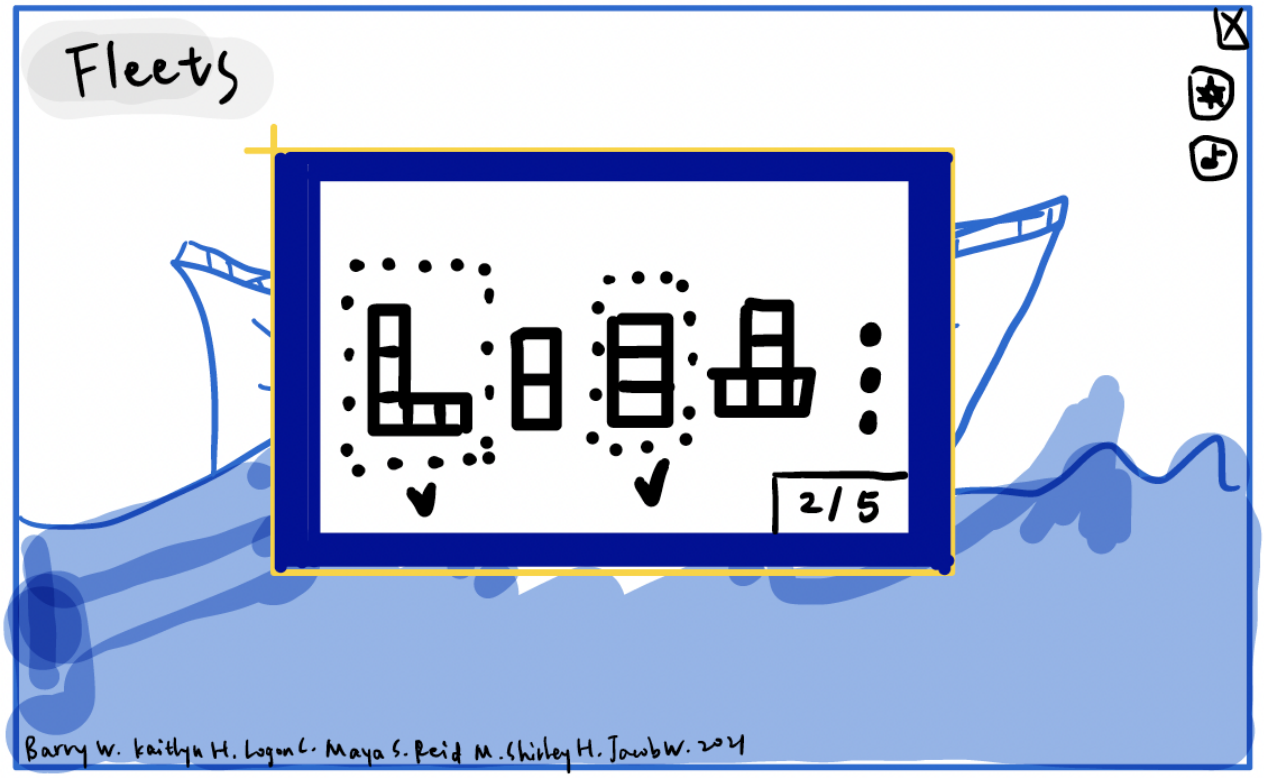


Figure 8: Fleet placement page

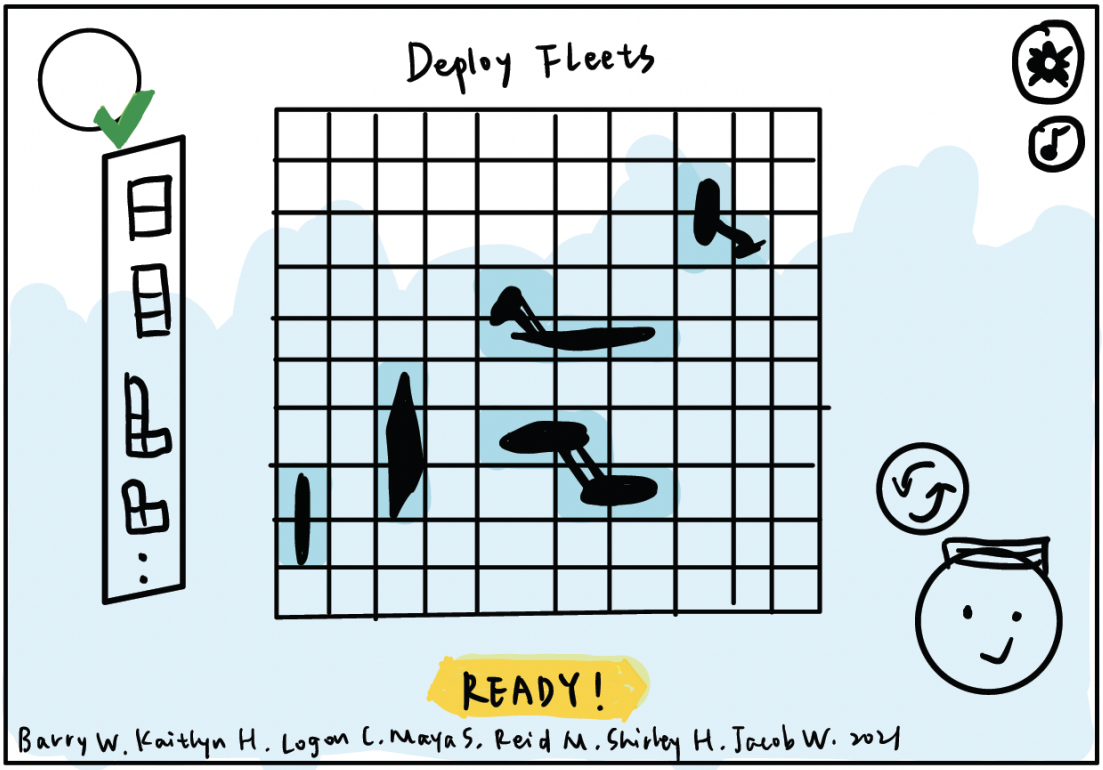


Figure 9: Player vs. player intermediary screen

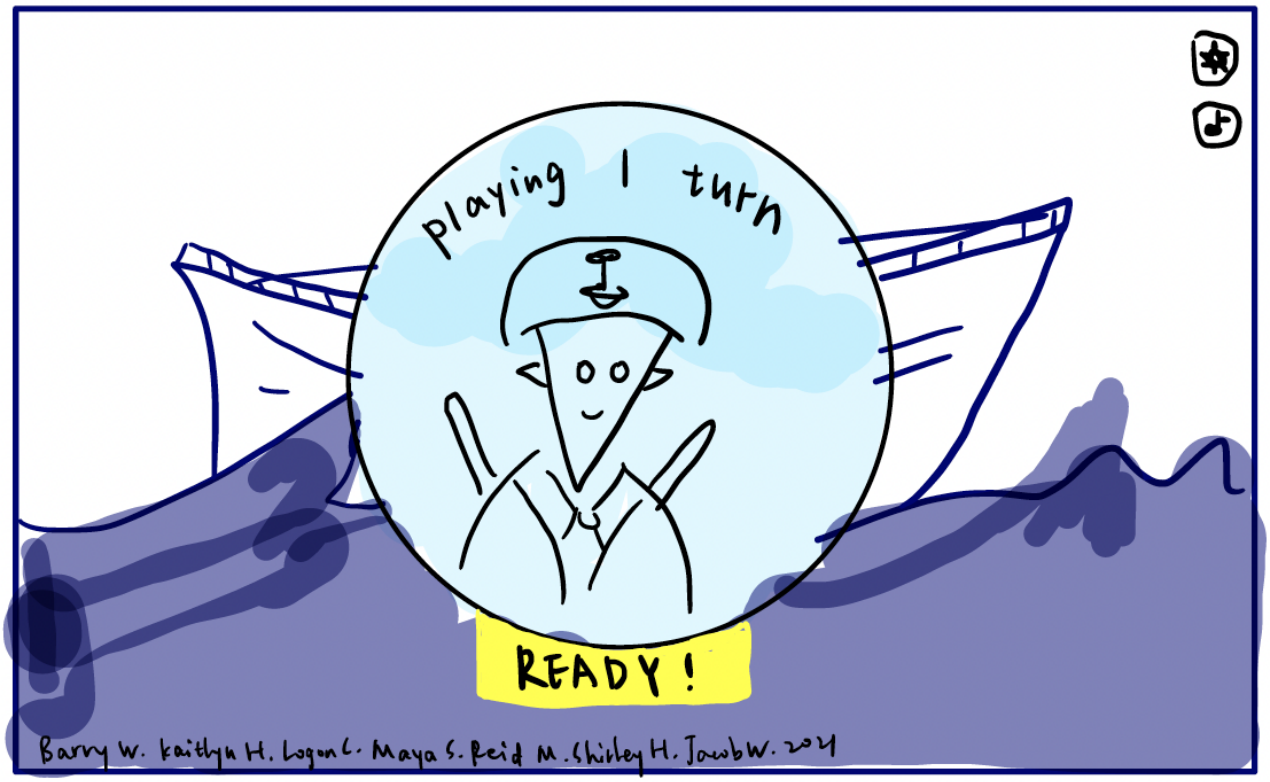


Figure 10: Main gameplay page

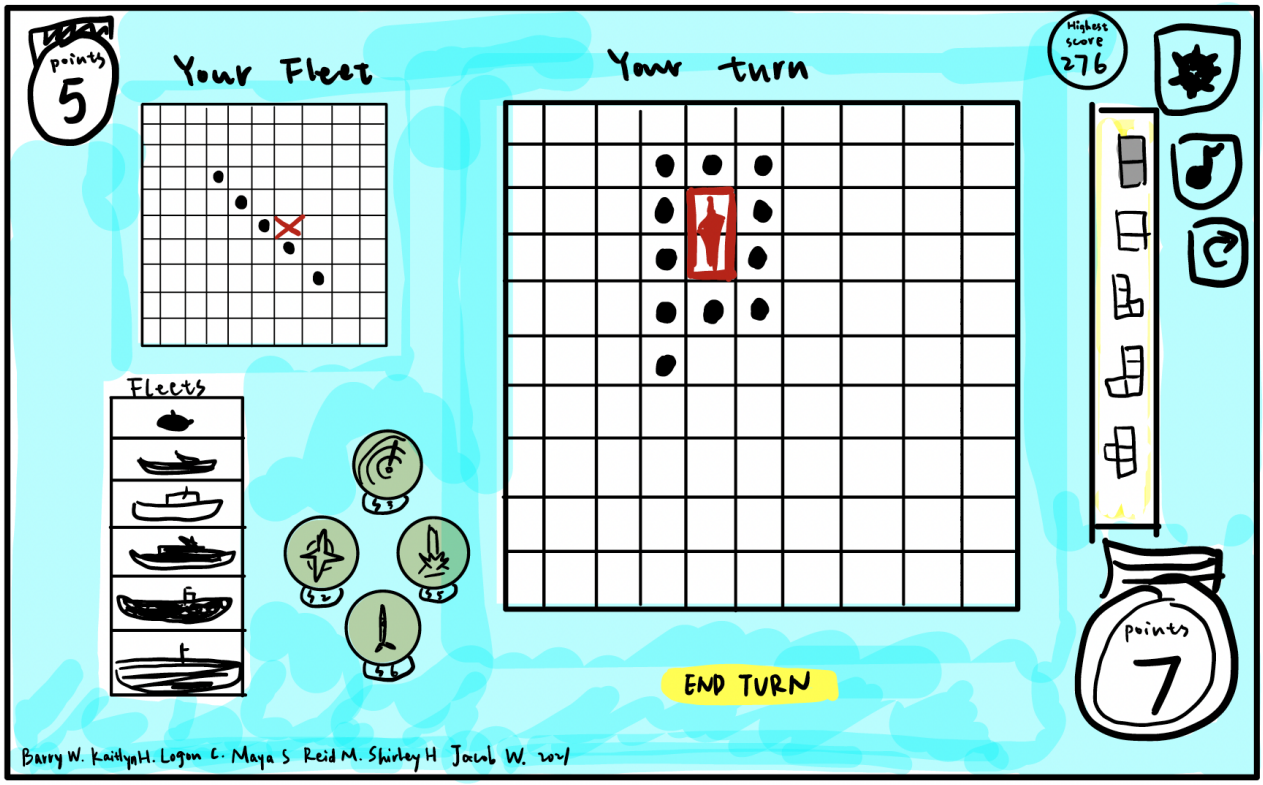
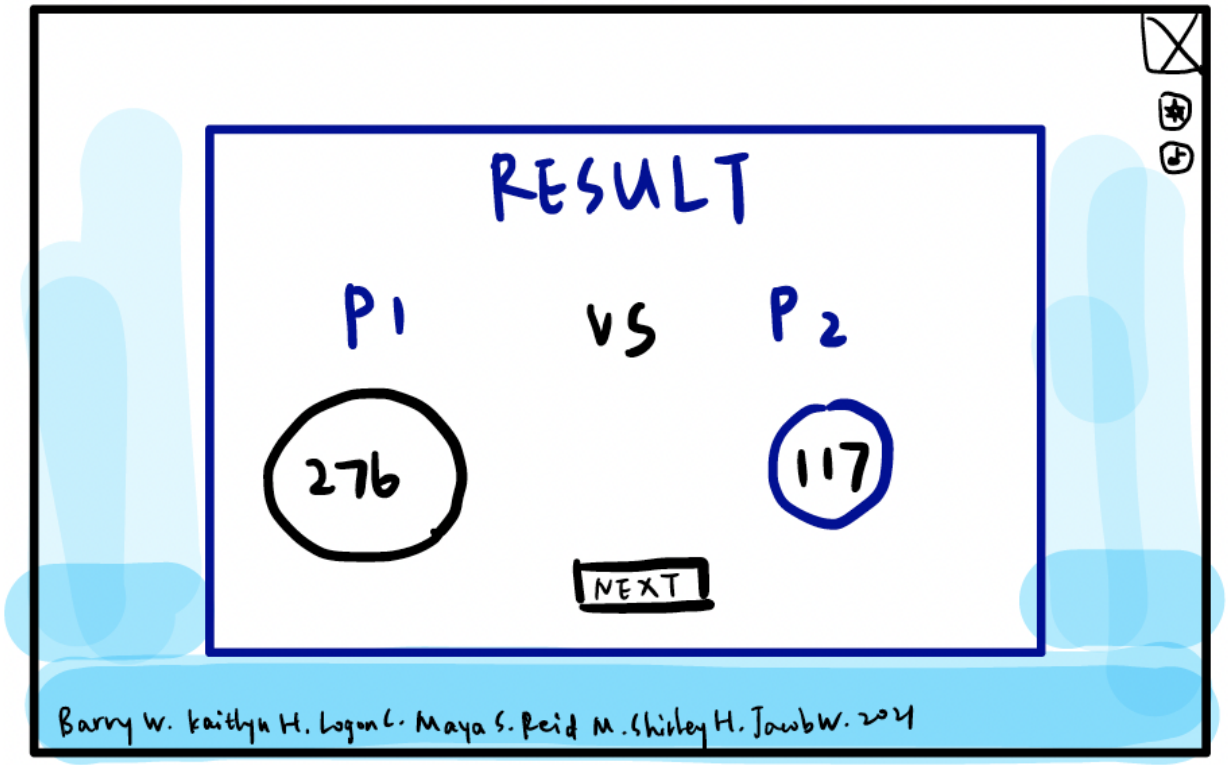


Figure 11: Game result popup



5. Team Roles

The following table lists all team members and their assigned roles.

Table 2: Team members and roles

Name	Role
Logan Copeland	Team Lead / Developer
Reid Moffat	Software Architect / Developer
Maya Steckly	Note-Taker / Developer
Kaitlyn Hung	Note-Taker / Developer
Shirley Hu	Graphic Designer / Developer
Jacob Wong	Developer
Baorong Wei	Developer